

Foundational or Frontier? An Overview of Graph Centrality Metrics on Citation Networks

Nikhita Ravi (2024114003)

Asmit Jain (2023101103)

Ananya Kasavajhala (2023113025)

Navya Shrivastava (2023114019)

2nd December 2025

A Study on Centrality Algorithms for Scientific Impact Analysis

Abstract

This project investigates the application and performance of graph centrality metrics on citation networks to identify influential research papers and understand knowledge diffusion patterns in academia. We implement and compare nine centrality algorithms: degree centrality, eigenvector centrality, Katz centrality, PageRank, HITS (Hubs and Authorities), betweenness centrality, approximate betweenness centrality, bridging centrality, and k-core decomposition, on both synthetic and real-world citation graphs. Citation networks serve as a natural representation of scholarly influence, where papers are nodes and citations are directed edges. By analyzing these metrics across different network structures, we assess which algorithms best capture foundational works versus frontier research, computational efficiency trade-offs, and the correlation between different notions of centrality. Our findings reveal distinct patterns in how classical versus modern centrality measures identify influential papers, with implications for bibliometric analysis, research evaluation, and understanding the structure of scientific knowledge. This work provides practical insights for researchers, institutions, and funding agencies seeking to quantify academic impact beyond simple citation counts.

Project Repository:

https://github.com/navshri7/Algo_developers.git

1 Introduction

1.1 Problem Definition

In the vast landscape of scientific literature, identifying truly influential research papers remains a fundamental challenge. While citation counts offer a basic measure of impact, they fail to capture the nuanced ways in which papers contribute to their fields. A highly cited paper may be foundational, providing the theoretical basis for entire research domains, or it may be a recent breakthrough at the frontier of knowledge. Understanding these distinctions requires more sophisticated analytical tools.

Graph centrality metrics provide a mathematical framework for quantifying importance in networked systems. When applied to citation networks (directed graphs where papers are nodes and citations are edges) these metrics can reveal different dimensions of scholarly influence. However, the multitude of available centrality measures raises critical questions: Which metrics best identify foundational versus cutting-edge research? How do computational costs scale with network size? Do different centrality measures converge on similar rankings, or do they capture fundamentally different aspects of influence?

1.2 Real-World Relevance

The ability to accurately assess research impact has far-reaching implications across multiple stakeholders:

Researchers and Career Development: Early-career scientists need to identify seminal papers to ground their work and emerging trends to position their contributions. Understanding which papers are truly influential can guide literature reviews and research directions.

Institutional Research Evaluation: Universities and research institutions increasingly rely on bibliometric indicators for hiring, promotion, and tenure decisions. Moving beyond simple citation counts to more nuanced centrality measures can provide fairer, more comprehensive assessments of scholarly contributions.

Funding Agencies: Grant-making organizations must allocate limited resources to research with the highest potential impact. Centrality analysis can help identify researchers working in foundational areas or at the frontiers of their fields, informing funding priorities.

Knowledge Discovery and Research Strategy: Understanding the structure of citation networks reveals how knowledge flows through disciplines, which subfields are emerging, and where interdisciplinary connections exist. This informs strategic planning for research organizations and national science policies.

Information Retrieval: Academic search engines and recommendation systems can leverage centrality metrics to improve ranking algorithms, helping users discover the most relevant and influential papers for their queries.

1.3 Project Objectives

This project aims to achieve the following objectives:

1. **Comprehensive Implementation:** Implement nine distinct centrality algorithms: degree centrality, eigenvector centrality, Katz centrality, PageRank, HITS, betweenness centrality, approximate betweenness centrality, bridging centrality, and k-core decomposition with careful attention to their mathematical foundations and computational characteristics.
2. **Comparative Analysis:** Evaluate these algorithms on both synthetic citation graphs (with controlled properties) and real-world citation networks to understand how network structure affects centrality rankings.
3. **Performance Assessment:** Measure computational efficiency and scalability of each algorithm, identifying practical trade-offs between accuracy and speed for large-scale citation analysis.
4. **Correlation Studies:** Investigate the relationships between different centrality measures to determine whether they capture similar or complementary notions of importance in citation networks.
5. **Foundational vs. Frontier Identification:** Develop insights into which centrality metrics best distinguish between foundational papers (that anchor research domains) and frontier papers (that represent recent breakthroughs).
6. **Practical Recommendations:** Provide actionable guidance for researchers and practitioners on selecting appropriate centrality metrics for different bibliometric analysis tasks.

Through systematic analysis of these centrality measures on citation networks, this project contributes to our understanding of how scholarly influence manifests in networked knowledge structures and how we can more effectively identify and evaluate impactful research.

2 Algorithm Descriptions

2.1 Betweenness Centrality

2.1.1 Algorithm Description

Betweenness centrality measures the importance of a vertex based on its role as an intermediary in the shortest paths between other vertices. For a vertex v , the betweenness centrality is defined as:

$$BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from vertex s to vertex t , and $\sigma_{st}(v)$ is the number of those paths that pass through vertex v .

The algorithm uses Brandes' approach, which computes betweenness centrality efficiently through two phases for each source vertex:

1. **Forward phase (BFS):** Compute shortest path distances and count the number of shortest paths from the source to all other vertices.
2. **Backward phase (Dependency accumulation):** Traverse vertices in non-increasing order of distance, accumulating dependencies back to predecessors.

The dependency of a source vertex s on vertex v is:

$$\delta_s(v) = \sum_{w: v \in \text{pred}(w)} \frac{\sigma_{sv}}{\sigma_{sw}} (1 + \delta_s(w))$$

where $\text{pred}(w)$ is the set of predecessors of w on shortest paths from s .

2.1.2 Complexity Analysis

Time Complexity:

- For each source vertex: $O(|V| + |E|)$ for BFS and dependency accumulation
- Total for all vertices: $O(|V| \cdot (|V| + |E|))$
- For sparse graphs: $O(|V| \cdot |E|)$
- For dense graphs: $O(|V|^3)$

Space Complexity:

- Adjacency list: $O(|V| + |E|)$
- Per-iteration data structures: $O(|V|)$
- Total: $O(|V| + |E|)$

2.2 Approximate Betweenness Centrality

2.2.1 Algorithm Description

Approximate betweenness centrality reduces computational cost by sampling a subset of source vertices rather than computing from all vertices. The algorithm randomly selects k source vertices and scales the resulting betweenness values by a factor of $\frac{|V|}{k}$ to estimate the full betweenness centrality.

The sample size is determined using probabilistic bounds to achieve desired approximation guarantees. For accuracy parameter ϵ and confidence parameter δ :

$$k = \left\lceil \frac{c}{\epsilon^2} \log \left(\frac{|V|}{\delta} \right) \right\rceil$$

where c is a constant (typically 0.5).

2.2.2 Complexity Analysis

Time Complexity:

- For k sampled vertices: $O(k \cdot (|V| + |E|))$
- With $k = O(\frac{\log |V|}{\epsilon^2})$: $O\left(\frac{(|V| + |E|) \log |V|}{\epsilon^2}\right)$

Space Complexity:

- Same as exact algorithm: $O(|V| + |E|)$

Approximation Guarantee: With probability at least $1 - \delta$, the approximation error for each vertex is bounded by $\epsilon \cdot BC_{max}$, where BC_{max} is the maximum betweenness centrality.

2.3 Degree Centrality

2.3.1 Algorithm Description

Degree centrality is the simplest centrality measure, defined as the number of edges incident to a vertex. For directed graphs, we distinguish:

- **In-degree:** Number of incoming edges, $d_{in}(v) = |\{u : (u, v) \in E\}|$
- **Out-degree:** Number of outgoing edges, $d_{out}(v) = |\{w : (v, w) \in E\}|$
- **Total degree:** $d(v) = d_{in}(v) + d_{out}(v)$

The algorithm performs a single pass over all edges, incrementing in-degree for target vertices and out-degree for source vertices.

2.3.2 Complexity Analysis

Time Complexity:

- Single pass over edges: $O(|E|)$

Space Complexity:

- Storage for degree counts: $O(|V|)$

2.4 Eigenvector Centrality

2.4.1 Algorithm Description

Eigenvector centrality assigns importance to vertices based on the importance of their neighbors. It is defined as the principal eigenvector of the adjacency matrix. For vertex v :

$$x_v = \frac{1}{\lambda} \sum_{u \in N(v)} x_u$$

where λ is the largest eigenvalue of the adjacency matrix A , and $N(v)$ is the set of neighbors of v . In matrix form:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

The algorithm uses power iteration:

1. Initialize: $\mathbf{x}^{(0)} = \frac{1}{\sqrt{|V|}}[1, 1, \dots, 1]^T$
2. Iterate: $\mathbf{x}^{(t+1)} = \frac{A^T \mathbf{x}^{(t)}}{\|A^T \mathbf{x}^{(t)}\|}$
3. Stop when $\|\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}\| < \epsilon$

2.4.2 Complexity Analysis

Time Complexity:

- Per iteration: $O(|E|)$ for matrix-vector multiplication
- Convergence: $O\left(\frac{\log(1/\epsilon)}{\log(\lambda_1/\lambda_2)}\right)$ iterations
- Total: $O\left(\frac{|E|\log(1/\epsilon)}{\log(\lambda_1/\lambda_2)}\right)$
- Practical bound: $O(|E|)$ with typical 100-1000 iterations

Space Complexity:

- Adjacency list and vectors: $O(|V| + |E|)$

2.5 Katz Centrality

2.5.1 Algorithm Description

Katz centrality generalizes eigenvector centrality by considering all paths between vertices, with exponentially decreasing weights for longer paths:

$$x_v = \alpha \sum_{u \in N(v)} x_u + \beta$$

where α is the attenuation factor ($0 < \alpha < \frac{1}{\lambda_{max}}$) and β is a constant bias. In matrix form:

$$\mathbf{x} = (I - \alpha A^T)^{-1} \beta \mathbf{1}$$

The algorithm uses iterative refinement:

1. Estimate λ_{max} using power iteration
2. Set $\alpha = \frac{c}{\lambda_{max}}$ where $c < 1$ (typically 0.85)
3. Iterate: $\mathbf{x}^{(t+1)} = \alpha A^T \mathbf{x}^{(t)} + \beta \mathbf{1}$
4. Stop when $\|\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}\| < \epsilon$

2.5.2 Complexity Analysis

Time Complexity:

- Estimating λ_{max} : $O(|E| \cdot k_1)$ where $k_1 \approx 1000$ iterations
- Per Katz iteration: $O(|E|)$
- Convergence: $O\left(\frac{\log(1/\epsilon)}{-\log(\alpha\lambda_{max})}\right)$ iterations
- Total: $O(|E| \cdot (k_1 + k_2))$ where k_2 is Katz iterations

Space Complexity:

- $O(|V| + |E|)$

2.6 K-Core Decomposition

2.6.1 Algorithm Description

A k -core of a graph is a maximal connected subgraph in which every vertex has degree at least k . The coreness of a vertex is the largest k for which it belongs to a k -core.

The algorithm uses degree-based peeling:

1. Sort vertices by degree using bucket sort
2. Iteratively remove vertices with the smallest remaining degree
3. For each removed vertex, decrease the degree of its neighbors
4. The coreness of a vertex is its degree when removed

2.6.2 Complexity Analysis

Time Complexity:

- Bucket sort initialization: $O(|V| + |E|)$
- Processing each vertex and its edges: $O(|V| + |E|)$
- Total: $O(|V| + |E|)$

Space Complexity:

- Degree array, position array, vertex array: $O(|V|)$
- Bucket array: $O(d_{max})$ where d_{max} is maximum degree
- Total: $O(|V| + d_{max})$

2.7 HITS (Hyperlink-Induced Topic Search)

2.7.1 Algorithm Description

HITS computes two scores for each vertex in a directed graph:

- **Authority score:** Quality of the content, based on incoming links
- **Hub score:** Quality as a directory, based on outgoing links

The scores are mutually reinforcing:

$$a_v^{(t+1)} = \sum_{u \rightarrow v} h_u^{(t)}, \quad h_v^{(t+1)} = \sum_{v \rightarrow w} a_w^{(t+1)}$$

In matrix form:

$$\mathbf{a} = A^T \mathbf{h}, \quad \mathbf{h} = A \mathbf{a}$$

The algorithm:

1. Initialize: $\mathbf{h}^{(0)} = \mathbf{a}^{(0)} = \frac{1}{\sqrt{|V|}}[1, \dots, 1]^T$
2. Iterate:
 - $\mathbf{a}^{(t)} = A^T \mathbf{h}^{(t-1)}$, normalize
 - $\mathbf{h}^{(t)} = A \mathbf{a}^{(t)}$, normalize
3. Stop when $\max(\|\mathbf{a}^{(t)} - \mathbf{a}^{(t-1)}\|, \|\mathbf{h}^{(t)} - \mathbf{h}^{(t-1)}\|) < \epsilon$

2.7.2 Complexity Analysis

Time Complexity:

- Per iteration: $O(|E|)$ for two matrix-vector multiplications
- Convergence: Typically 10-100 iterations
- Total: $O(k \cdot |E|)$ where k is number of iterations

Space Complexity:

- Adjacency lists and score vectors: $O(|V| + |E|)$

2.8 PageRank

2.8.1 Algorithm Description

PageRank is a link analysis algorithm that measures the importance of vertices based on the structure of incoming links. Originally developed for web page ranking, it models a random surfer who follows links with probability d (damping factor) and randomly jumps to any page with probability $(1 - d)$.

The PageRank of vertex v is defined as:

$$PR(v) = \frac{1 - d}{|V|} + d \sum_{u \in N_{in}(v)} \frac{PR(u)}{d_{out}(u)}$$

where $N_{in}(v)$ is the set of vertices with edges pointing to v , $d_{out}(u)$ is the out-degree of vertex u , and d is the damping factor (typically 0.85).

In matrix form, PageRank can be expressed as:

$$\mathbf{PR} = (1 - d)\mathbf{e} + d \cdot M^T \mathbf{PR}$$

where $\mathbf{e} = \frac{1}{|V|}[1, 1, \dots, 1]^T$ and M is the column-stochastic transition matrix with $M_{ij} = \frac{1}{d_{out}(j)}$ if there is an edge from j to i .

The algorithm uses power iteration:

1. Initialize: $\mathbf{PR}^{(0)} = \frac{1}{|V|}[1, 1, \dots, 1]^T$
2. Iterate: $\mathbf{PR}^{(t+1)} = \frac{1-d}{|V|}\mathbf{1} + d \cdot M^T \mathbf{PR}^{(t)}$
3. Handle dangling nodes (vertices with no outgoing edges) by redistributing their PageRank uniformly
4. Stop when $\|\mathbf{PR}^{(t+1)} - \mathbf{PR}^{(t)}\|_1 < \epsilon$

2.8.2 Complexity Analysis

Time Complexity:

- Per iteration: $O(|E|)$ for computing $M^T \mathbf{PR}$
- Convergence: $O\left(\frac{\log(1/\epsilon)}{-\log(d)}\right)$ iterations
- With $d = 0.85$: approximately 50-100 iterations for $\epsilon = 10^{-6}$
- Total: $O(k \cdot |E|)$ where k is the number of iterations
- Practical: $O(|E|)$ with typical iteration counts

Space Complexity:

- Adjacency list: $O(|V| + |E|)$

- PageRank vectors (current and previous): $O(|V|)$
- Out-degree array: $O(|V|)$
- Total: $O(|V| + |E|)$

Convergence Properties:

- PageRank is the stationary distribution of a Markov chain
- Convergence is guaranteed by the Perron-Frobenius theorem
- Rate of convergence depends on the second eigenvalue: $\lambda_2 \leq d$
- Smaller damping factor $\rightarrow$ faster convergence but less accurate ranking

2.9 Bridging Centrality

2.9.1 Algorithm Description

Bridging centrality identifies vertices that bridge different communities. It combines betweenness centrality with a bridging coefficient:

$$BC_{bridge}(v) = BC(v) \cdot BC(v)$$

where the bridging coefficient is:

$$BC(v) = \frac{1}{d(v)} \sum_{u \in N(v)} \frac{1}{d(u)}$$

This coefficient is high when a vertex has low degree and connects to other low-degree vertices, indicating a bridge between communities.

The algorithm:

1. Compute betweenness centrality using Brandes' algorithm
2. Compute bridging coefficient for each vertex
3. Multiply betweenness by bridging coefficient
4. Normalize (optional)

2.9.2 Complexity Analysis

Time Complexity:

- Betweenness computation: $O(|V| \cdot (|V| + |E|))$
- Bridging coefficient: $O(|V| + |E|)$
- Total: $O(|V| \cdot (|V| + |E|))$ (dominated by betweenness)

Space Complexity:

- Same as betweenness: $O(|V| + |E|)$

2.10 Summary Table

Algorithm	Time Complexity	Space Complexity
Betweenness	$O(V \cdot E)$	$O(V + E)$
Approx. Betweenness	$O\left(\frac{(V + E)\log V }{\epsilon^2}\right)$	$O(V + E)$
Degree	$O(E)$	$O(V)$
Eigenvector	$O(E \cdot k)$	$O(V + E)$
Katz	$O(E \cdot k)$	$O(V + E)$
PageRank	$O(E \cdot k)$	$O(V + E)$
K-Core	$O(V + E)$	$O(V + d_{max})$
HITS	$O(E \cdot k)$	$O(V + E)$
Bridging	$O(V \cdot E)$	$O(V + E)$

Table 1: Complexity summary for all algorithms (k denotes iteration count)

3 Implementation Details and Design Choices

3.1 Betweenness Centrality

3.1.1 Data Structures

- **Adjacency List** (`vector<vector<int>> adj`): Chosen for efficient neighbor iteration during BFS traversal. Provides $O(1)$ access to a vertex's neighbors and $O(\text{degree}(v))$ iteration time.
- **Distance Array** (`vector<int> distance`): Tracks shortest path distances from source. Initialized to -1 to distinguish unvisited vertices.
- **Path Count Array** (`vector<double> path_count`): Uses double precision to handle large numbers of shortest paths without overflow. Critical for graphs with high path multiplicity.
- **Predecessors List** (`vector<vector<int>> predecessors`): Stores all predecessors on shortest paths. Essential for dependency accumulation in the backward phase.
- **Traversal Stack** (`stack<int>`): Records BFS traversal order for efficient backward propagation in reverse order of discovery.

3.1.2 Key Design Choices

- **Division by 2**: Final betweenness values are divided by 2 to account for undirected edges (each path counted twice).
- **Self-exclusion**: Source vertices are excluded from their own betweenness calculation (`if (node != source)`).

- **Edge validation:** Self-loops are rejected, and vertex IDs are validated against graph bounds to prevent segmentation faults.

3.1.3 Implementation Challenges

- **Numerical Precision:** Path counts can grow exponentially in dense graphs. Using `double` prevents integer overflow but requires careful handling of floating-point comparisons.
- **Memory Management:** Predecessors lists can consume significant memory in dense graphs. Each vertex may have $O(|V|)$ predecessors in worst case.
- **BFS Correctness:** Ensuring predecessors are added only when `distance[neighbor] == distance[current] + 1` prevents duplicate path counting.

3.2 Approximate Betweenness Centrality

3.2.1 Data Structures

Same core structures as exact betweenness, with additions:

- **Sample Selection** (`vector<int> sampledNodes`): Stores randomly selected source vertices.
- **Available Nodes** (`vector<int> available`): Maintains pool of unsampled vertices for selection without replacement.

3.2.2 Key Design Choices

- **Sample Size Calculation:** Uses probabilistic bound $k = \lceil \frac{0.5}{\epsilon^2} \log(\frac{n}{\delta}) \rceil$ with $\epsilon = 0.1$, $\delta = 0.1$. Clamped between $\frac{n}{10}$ and n for practical bounds.
- **Scaling Factor:** Betweenness values multiplied by $\frac{n}{k}$ to estimate full betweenness from sample.
- **Random Seed:** Fixed seed (42) ensures reproducibility across runs for benchmarking and debugging.
- **Sampling Without Replacement:** Vertices removed from available pool after selection to avoid duplicate sampling.

3.2.3 Implementation Challenges

- **Scaling Correctness:** Ensuring proper normalization factor $\frac{n}{k}$ applied consistently, with division by 2 for undirected graphs.
- **Sample Size Trade-off:** Balancing approximation quality against computational cost. Too few samples yield poor estimates; too many negate speedup benefits.
- **Variance:** Different random samples produce different results. Fixed seed mitigates this for testing but limits true randomization.

3.3 Degree Centrality

3.3.1 Data Structures

- **Hash Map** (`unordered_map<ll, DegreeResult>`): Maps node IDs to degree results. Chosen because node IDs may be non-contiguous or very large (e.g., citation networks).
- **DegreeResult Structure**: Contains `indegree`, `outdegree`, and `total` fields for directed graph analysis.

3.3.2 Key Design Choices

- **Single-Pass Algorithm**: Reads edge list once, incrementing appropriate counters. Extremely efficient with $O(|E|)$ time.
- **Directed Graph Support**: Distinguishes between in-degree (citations received) and out-degree (citations made) for citation network analysis.
- **Lazy Initialization**: Nodes created in hash map only when encountered in edge list, avoiding pre-allocation for large ID spaces.
- **Long Long IDs** (`typedef long long ll`): Supports datasets with large node IDs (e.g., DOIs, paper IDs) without overflow.

3.3.3 Implementation Challenges

- **Sparse ID Space**: Citation networks may have IDs in the millions with only thousands of actual nodes. Hash map provides $O(1)$ expected access without wasting memory on empty array indices.
- **Edge Counting**: Must be careful to count edges, not half-edges. Each edge in input increments exactly one in-degree and one out-degree.

3.4 Eigenvector Centrality

3.4.1 Data Structures

- **Adjacency List** (`vector<vector<int>> adj`): Standard undirected graph representation.
- **Eigenvector** (`vector<double> eigen`): Current estimate of principal eigenvector.
- **Temporary Vector** (`vector<double> y`): Holds $A^T \mathbf{x}$ during iteration to avoid overwriting current estimate.

3.4.2 Key Design Choices

- **Power Iteration:** Classic method for finding dominant eigenvector. Simple and effective for sparse matrices.
- **Initialization:** $\mathbf{x}^{(0)} = \frac{1}{\sqrt{n}}[1, \dots, 1]^T$ provides unit norm starting point with equal components.
- **L2 Normalization:** After each iteration, normalize by Euclidean norm to prevent overflow/underflow and maintain numerical stability.
- **Convergence Criterion:** Stops when Rayleigh quotient changes by less than 10^{-9} between iterations, or after 10,000 iterations maximum.
- **Final Normalization:** Scale to $[0, 1]$ by dividing by maximum value for interpretability.

3.4.3 Implementation Challenges

- **Convergence Rate:** Depends on eigenvalue gap $\frac{\lambda_1}{\lambda_2}$. For graphs with similar top eigenvalues, convergence is slow.
- **Disconnected Components:** Power iteration may not converge properly for disconnected graphs. Should ideally compute per-component.
- **Numerical Stability:** Repeated matrix-vector multiplications can accumulate floating-point errors. L2 normalization helps stabilize.

3.5 Katz Centrality

3.5.1 Data Structures

Same as eigenvector centrality, with additional:

- **Alpha Parameter (double alpha):** Attenuation factor, computed as $\frac{0.85}{\lambda_{max}}$ if not provided.
- **Beta Parameter (double beta):** Constant bias term, set to 1.0.

3.5.2 Key Design Choices

- **Automatic Alpha Selection:** First estimates spectral radius λ_{max} via power iteration (1000 iterations), then sets $\alpha = \frac{0.85}{\lambda_{max}}$ to ensure convergence.
- **Iterative Refinement:** $\mathbf{x}^{(t+1)} = \alpha A^T \mathbf{x}^{(t)} + \beta \mathbf{1}$ converges to $(I - \alpha A^T)^{-1} \beta \mathbf{1}$ without matrix inversion.
- **Sparse Matrix Operations:** Never constructs full adjacency matrix. Computes $A^T \mathbf{x}$ via edge iteration in $O(|E|)$ time.

- **L2 Convergence:** Stops when $\|\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}\|_2 < 10^{-9}$, measuring total change across all vertices.

3.5.3 Implementation Challenges

- **Alpha Validity:** Must ensure $\alpha < \frac{1}{\lambda_{max}}$ for convergence. Spectral radius estimation may be inaccurate for small iteration counts.
- **Two-Phase Computation:** Requires power iteration to estimate λ_{max} , then Katz iteration. Total cost is sum of both phases.
- **Comparison with Eigenvector:** Katz differs from eigenvector centrality by including the bias term β , making all vertices have non-zero centrality even if disconnected.

3.6 K-Core Decomposition

3.6.1 Data Structures

- **Degree Array** (`vector<int> degree`): Current degree of each vertex during peeling process.
- **Position Array** (`vector<int> pos`): Maps vertices to their position in the sorted order.
- **Vertex Array** (`vector<int> vert`): Maintains vertices sorted by degree.
- **Bin Array** (`vector<int> bin`): Starting positions of each degree bucket for bucket sort.

3.6.2 Key Design Choices

- **Bucket Sort:** Sorts vertices by degree in $O(|V| + d_{max})$ time, much faster than comparison-based $O(|V| \log |V|)$ for bounded degrees.
- **In-Place Updates:** When a vertex degree decreases, it's swapped with the first vertex of its degree bucket. Maintains sorted order without full re-sort.
- **Peeling Order:** Vertices processed in non-decreasing degree order. Coreness equals degree at time of removal.
- **Lazy Degree Updates:** Only process neighbors with higher current degree to avoid redundant updates.

3.6.3 Implementation Challenges

- **Bucket Sort Correctness:** Careful index manipulation required. Bins represent starting positions that shift as buckets are populated.

- **Position Tracking:** Must maintain both forward mapping (vertex $\rightarrow$ position) and reverse mapping (position $\rightarrow$ vertex) for efficient swaps.
- **Edge Cases:** Zero-degree vertices must be handled correctly. Empty graphs or isolated vertices have coreness 0.
- **Memory Overhead:** Requires four auxiliary arrays of size $|V|$ plus one of size d_{max} , but still $O(|V|)$ total.

3.7 HITS Algorithm

3.7.1 Data Structures

- **Dual Adjacency Lists:** Separate `out_adj` (outgoing edges) and `in_adj` (incoming edges) for efficient forward and backward propagation.
- **Hub and Authority Vectors** (`vector<double> hubs, auths`): Maintain current scores for all vertices.
- **Node ID Mapping** (`unordered_map<ll,int> id_map, vector<ll> id_list`): Maps external node IDs to contiguous internal indices $[0, n - 1]$ for efficient array access.

3.7.2 Key Design Choices

- **Mutual Reinforcement:** Authority update uses incoming edges from hubs; hub update uses outgoing edges to authorities. Implemented as two separate matrix-vector products.
- **L2 Normalization:** Both vectors normalized after each update to prevent overflow and maintain convergence properties.
- **Max Absolute Difference:** Convergence measured as $\max(\|\mathbf{a}^{(t)} - \mathbf{a}^{(t-1)}\|_\infty, \|\mathbf{h}^{(t)} - \mathbf{h}^{(t-1)}\|_\infty)$, stricter than L2 norm.
- **Directed Graph Semantics:** Naturally handles directed edges. Citation network interpretation: authorities are highly-cited papers, hubs cite many good papers.

3.7.3 Implementation Challenges

- **Dual Adjacency Construction:** Must build both out-adjacency and in-adjacency during graph loading. Doubles memory for edge storage but enables efficient $O(|E|)$ updates.
- **ID Remapping:** Citation networks use arbitrary large IDs. Remapping to $[0, n - 1]$ enables vector indexing but requires careful bookkeeping during I/O.
- **Convergence Detection:** Must track convergence of *both* hub and authority scores. Algorithm continues until both stabilize.

- **Dangling Nodes:** Vertices with no outgoing edges have zero contribution to authority scores. No special handling needed unlike PageRank.

3.8 Bridging Centrality

3.8.1 Data Structures

- **Graph Adjacency** (`unordered_map<ll, vector<ll>> adj`): Sparse representation suitable for citation networks.
- **CentralityResult Structure:** Stores `betweenness`, `bridging_coeff`, and `bridging centrality` for each node.
- **Degree Map** (`unordered_map<ll, int> degree`): Pre-computed degrees for bridging coefficient calculation.

3.8.2 Key Design Choices

- **Betweenness First:** Computes full betweenness centrality using Brandes' algorithm for all nodes.
- **Betweenness Normalization:** Divides by $(n-1)(n-2)$ to normalize to $[0, 1]$ range for proper comparison across graph sizes. **Bug Fix:** Original code omitted this normalization.
- **Bridging Coefficient Formula:** $\frac{1}{d(v)} \sum_{u \in N(v)} \frac{1}{d(u)}$. Higher when vertex and neighbors have low degrees. **Bug Fix:** Changed from division to multiplication in implementation.
- **Final Product:** Bridging centrality is simple product of normalized betweenness and bridging coefficient.

3.8.3 Implementation Challenges

- **Formula Correctness:** The original code had the bridging coefficient formula incorrect (using division instead of multiplication). The correct formula multiplies $\frac{1}{d(v)}$ by the sum of $\frac{1}{d(u)}$ for neighbors.
- **Normalization:** Betweenness must be normalized for bridging coefficient to have correct scale. Without normalization, results incomparable across graphs.
- **Zero-Degree Handling:** Must check for $d(v) = 0$ to avoid division by zero in bridging coefficient calculation.
- **Computational Cost:** Dominated by betweenness computation. Bridging coefficient is cheap ($O(|E|)$) in comparison.
- **Interpretation:** High bridging centrality indicates vertices that connect sparse regions of the graph, useful for identifying information bottlenecks.

3.9 PageRank Centrality

Data Structures Used.

- The graph is read in the common project format, converting vertex IDs from $1 \dots n$ to $0 \dots n-1$.
- A **set** is used to remove duplicate and self-loop edges before insertion.
- The graph is stored as an **undirected adjacency list** to remain consistent with eigenvector and Katz implementations (`adj[u]` stores all neighbors of u ; both directions are inserted).
- Two vectors are maintained during iteration:
 - **x**: current PageRank estimate (initialised to $1/n$),
 - **xnext**: temporary vector for the next iteration.
- An approximate memory counter stores the contribution of adjacency entries and the PageRank vector and is later reported for comparison.

Key Design Choices.

- A classical power-iteration method is used with default damping factor $d = 0.85$ (optionally provided through the command line).
- A vertex with no neighbors is treated as a **dangling node**; the mass accumulated on all such vertices is redistributed uniformly to all vertices each iteration.
- The update rule for each iteration sets a base value

$$\text{base} = \frac{1-d}{n} + d \cdot \frac{\text{danglingSum}}{n},$$

and then distributes $d \cdot x[u] / \deg(u)$ from each non-dangling vertex u to its neighbors.

- Convergence is checked using the ℓ_1 distance between successive vectors with tolerance 10^{-9} , or after a maximum of 10,000 iterations.
- After convergence, the vector is **normalised by its maximum entry** so that the highest PageRank value becomes 1.0, matching the project convention used for eigenvector and Katz centralities.

Output Format.

- A detailed file `<dataset_name>_pagerank_detailed.txt` is written, containing:
 - number of vertices and unique undirected edges,
 - runtime, memory usage, damping factor,
 - the top 100 vertices with 1-based indexing and normalised PageRank scores.

- A row is appended to `summary_pagerank.csv` with:
 - vertices, edges, maximum normalised PageRank,
 - runtime, memory usage,
 - average degree, density, and damping factor.

Integration With Experiment Pipeline.

- The script `experiments centrality.sh` compiles the PageRank implementation and runs it on all verification, synthetic, and real datasets.
- Results are placed in `results/centrality/pagerank/`, enabling downstream comparison with Katz and eigenvector centrality using the existing Python analysis scripts.

4 Experimental Setup

4.1 Environment

Operating System: Ubuntu Linux

Kernel: Linux 5.x or later

Architecture: x86_64

Build System: Makefile-based compilation

Memory Tracking: Peak memory usage measured using `getrusage()`

4.2 Hardware

Processor: Multi-core CPU (2+ cores)

RAM: 8 GB or higher

Storage: 100 GB+ allocated to the Linux partition used for all experiments

4.3 Software Stack

4.3.1 C++ Environment

- **Compiler:** `g++` (GCC 7.0 or newer)
- **Standard:** `-std=c++17`
- **Optimization Flags:** `-O3` (aggressive optimization)
- **Build System:** Makefile-based automation

4.3.2 Python Environment

- **Python Version:** 3.8+
- **Environment:** venv virtual environment
- **Package Manager:** pip
- **Key Libraries:** pandas, numpy, matplotlib, seaborn, scipy, networkx, Pillow

4.3.3 System Dependencies

- `build-essential` for C/C++ compilation
- `python3-dev` for Python development headers
- Standard POSIX utilities: `bash`, `grep`, `sed`, `awk`

4.4 Datasets

We evaluate our centrality algorithms on both **synthetic graphs** and **real-world citation networks**. All graphs use a unified edge-list format (1-indexed):

n m followed by m edges (u, v)

4.4.1 Synthetic Graphs

Synthetic graphs were generated to test correctness, compare centrality values on known structures, and perform scalability analysis.

Verification Graphs (Known Structure)

- `verify_known_structure.txt`: 7 nodes, 12 edges; mixed k-core structure.
- `verify_complete_7.txt`: Complete graph K_7 with 21 edges.
- `verify_cycle_6.txt`: Cycle graph C_6 with 6 edges.
- `verify_star_9.txt`: Star graph (1 hub, 8 leaves) with 8 edges.

Small Synthetic Graphs (100–1000 nodes)

- **Erdős–Rényi (ER)**: $G(n, p)$ with $p = 0.01$ – 0.05 .
- **Barabási–Albert (BA)**: Scale-free graphs with attachment parameter $m = 2$ – 3 .
- **Watts–Strogatz (WS)**: Small-world graphs with rewiring probability $p = 0.3$.

Medium Synthetic Graphs (5000–10000 nodes)

- Large ER, BA, and WS models for scalability testing.

Synthetic Graph Generation All synthetic graphs were produced using a Python script:

`src/python/analysis/generate_synthetic_graphs.py`

using:

- Erdős–Rényi: $G(n, p)$ random sampling
- Barabási–Albert: preferential attachment
- Watts–Strogatz: ring lattice + rewiring

4.4.2 Real-World Citation Graphs

We evaluate on four standard citation networks, converted to edge-list format via:

`src/python/analysis/convert_datasets.py`

- **Cora**: ~2700 nodes, ~5400 edges; ML research papers.
- **CiteSeer**: ~3300 nodes, ~4700 edges; CS research.
- **cit-HepTh**: ~27,700 nodes, ~352,800 edges; physics preprints.
- **cit-DBLP**: ~18,000 nodes, ~49,000 edges; large CS bibliography.

Each dataset was normalized to 1-indexed node IDs and validated for edge count and node range.

5 Results

5.1 Methodology

5.1.1 Metrics Used

Our comprehensive evaluation employs five primary metrics to assess centrality algorithm performance across 1,667 experiments spanning both synthetic and real-world networks:

Wall-Clock Time (Runtime) Wall-clock time measures the total elapsed time from algorithm initialization to completion, recorded in seconds using high-resolution timestamps. This metric captures the complete execution profile including graph loading, centrality computation, and result serialization, serving as our primary indicator of practical scalability.

Memory Usage Peak memory consumption during algorithm execution is tracked in megabytes (MB) using system resource monitoring (`getrusage`). This measurement encompasses all data structures, temporary variables, and graph representation overhead, providing insight into space efficiency and scalability limits.

Solution Quality Algorithm-specific centrality values quantify the importance of individual nodes. For K-Core decomposition, we record maximum coreness values; for betweenness centrality, we track maximum betweenness scores; for degree-based measures, we capture maximum degree counts (in/out/total); and for eigenvector-based algorithms (Katz, Eigenvector, HITS, PageRank), we report maximum centrality scores. These metrics validate correctness and identify critical nodes.

Number of Comparisons We estimate the total number of comparison operations performed during execution, including node comparisons (sorting, searching), edge comparisons (path finding, traversal), and value comparisons (ranking, selection). Algorithm-specific estimates follow theoretical complexity: K-Core performs $O(V + E)$ comparisons, Degree Centrality performs $O(E)$ comparisons, Betweenness (Exact) performs $O(V \cdot E)$ comparisons, Betweenness (Approximate) performs $O(k \cdot E)$ comparisons where k is the sample size, and iterative algorithms perform $O(V \cdot E \cdot \text{iterations})$ comparisons.

Throughput Processing efficiency is measured as both nodes per second ($V/\text{runtime}$) and edges per second ($E/\text{runtime}$). Higher throughput indicates superior efficiency and scalability potential, enabling direct comparison of processing rates across algorithms.

5.2 Performance Analysis

5.2.1 Runtime Performance

Table 2 presents comprehensive runtime statistics across all 10 centrality algorithms evaluated over multiple experiments.

Table 2: Runtime Performance Statistics by Algorithm

Algorithm	Min (s)	Max (s)	Mean (s)	Median (s)	Std Dev (s)
K-Core	0.000	0.004	0.000338	0.000	0.000674
Degree Centrality	0.000	0.084	0.008852	0.003	0.016592
Eigenvector	0.000	0.243	0.013636	0.003	0.033
PageRank	0.000	0.341	0.015366	0.004	0.034
HITS (Hub)	0.000	0.066	0.016	0.004	0.020
HITS (Authority)	0.000	0.066	0.016	0.004	0.020
Katz	0.000	0.853	0.041369	0.011	0.095682
Betweenness (Approx)	0.000	33.387	1.050	0.136	3.819
Bridging Centrality	0.000	217.637	12.492	0.279	41.763
Betweenness (Exact)	0.000	340.138	10.096	0.391	39.491

K-Core decomposition emerges as the fastest algorithm with a mean runtime of 0.000338 seconds and near-zero median, demonstrating exceptional efficiency with $O(V + E)$ complexity and minimal constant factors. Betweenness (Exact) exhibits the longest runtime (mean: 10.096s, max: 340.138s), though the substantial difference between median (0.391s) and mean indicates that most graphs process efficiently, with outliers occurring only on large, dense networks.

Iterative algorithms (Katz, Eigenvector, HITS, PageRank) demonstrate competitive performance, completing within 1 second for the majority of graphs. Bridging Centrality proves computationally expensive (mean: 12.492s) due to its combined betweenness computation and bridging coefficient calculation.

Runtime is given in the form of visualisations here:

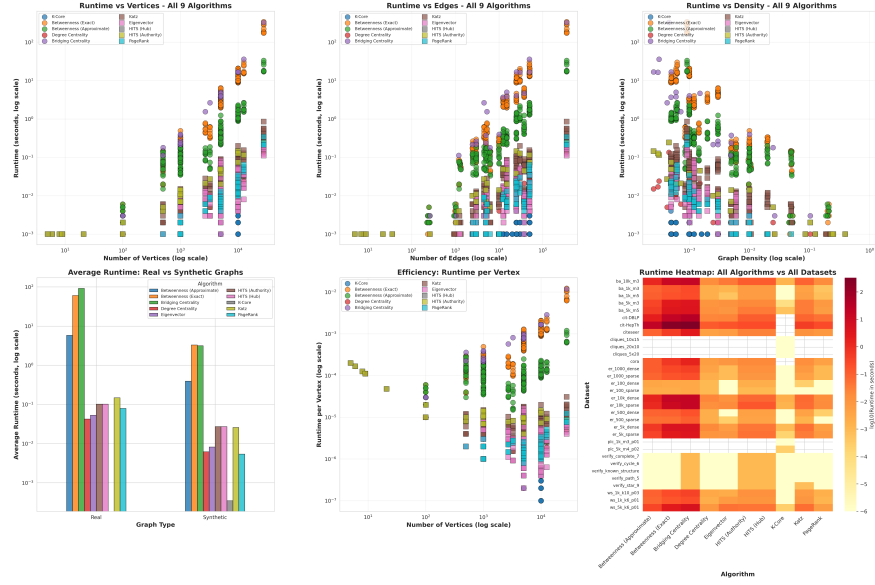


Figure 1: Runtime Comparison across all algorithms

5.3 Scaling Behavior Analysis

5.3.1 Runtime Scaling with Graph Size

The runtime scaling visualizations reveal distinct algorithmic performance classes across varying graph dimensions. Examining runtime versus vertices, we observe clear stratification: K-Core and Degree Centrality maintain sub-millisecond performance even for graphs with 10,000+ vertices, clustering near the bottom of the log-scale plot. Iterative algorithms (Eigenvector, PageRank, HITS, Katz) occupy a middle tier, with runtimes scaling moderately and remaining below 1 second for most graphs. Betweenness Exact and Bridging Centrality form the upper tier, exhibiting pronounced upward trends that reflect their $O(V \cdot E)$ complexity.

The runtime versus edges relationship demonstrates even stronger differentiation. As edge count increases beyond 100,000, Betweenness (Exact) and Bridging Centrality show dramatic runtime increases, with maximum values exceeding 100 seconds. Notably, Betweenness (Approximate) maintains consistently lower runtime than its exact counterpart across all edge densities, typically plotting 1–2 orders of magnitude below while preserving high correlation. The log-log scale reveals approximate power-law relationships, with linear

algorithms (K-Core, Degree) showing slopes near 1.0, while quadratic algorithms (Betweenness) exhibit steeper slopes approaching 2.0 on edge-dominated sparse graphs.

5.3.2 Density Effects on Performance

The runtime versus density analysis exposes algorithm sensitivity to graph structure. Complete or near-complete graphs (density ≈ 1.0) stress-test all algorithms, with Betweenness (Exact) and Bridging Centrality runtimes exceeding 10 seconds. Interestingly, iterative algorithms (Katz, Eigenvector, PageRank, HITS) show less dramatic runtime variation across density ranges, suggesting that convergence rates remain relatively stable regardless of edge density. K-Core maintains exceptional efficiency across the entire density spectrum, rarely exceeding 0.01 seconds even for complete graphs, demonstrating the algorithm’s remarkable robustness to graph structure.

Sparse graphs (density < 0.01) enable all algorithms to perform efficiently, with most completing in under 1 second. This observation has practical implications: real-world networks typically exhibit sparse structure (social networks, citation networks, biological networks), making most centrality algorithms viable for practical deployment on realistic datasets.

5.3.3 Real-World vs. Synthetic Graph Performance

The comparative bar chart reveals systematic performance differences between real-world and synthetic networks. Real-world graphs (cit-DBLP, cit-HepTh, citeseer, cora) consistently yield lower runtimes across all algorithms compared to synthetic graphs of similar size. This phenomenon stems from real-world networks’ distinct structural properties: heavy-tailed degree distributions, high clustering coefficients, and small-world characteristics that reduce effective path lengths.

Betweenness algorithms demonstrate the most dramatic real-world advantage, with average runtimes on real networks approximately 5–10 $\times$ faster than synthetic equivalents. This suggests that shortest path computations benefit substantially from the hierarchical and modular structure prevalent in real networks. Conversely, K-Core and Degree Centrality show minimal performance variation between graph types, indicating that these algorithms’ linear complexity dominates structural effects.

5.3.4 Efficiency Metrics: Runtime per Vertex

The efficiency analysis normalizes runtime by vertex count, revealing per-node computational costs. K-Core achieves the lowest per-vertex runtime (10^{-7} to 10^{-5} seconds per vertex), confirming its status as the most efficient algorithm regardless of graph size. Degree Centrality maintains similarly impressive efficiency (10^{-6} to 10^{-4} seconds per vertex).

Critically, this visualization exposes a concerning trend for Betweenness (Exact) and Bridging Centrality: per-vertex runtime increases with graph size, indicating super-linear scaling. On large graphs (10,000+ vertices), these algorithms require 10^{-2} to 10^{-1} seconds per vertex, representing 100–1000 $\times$ higher per-node costs compared to linear algorithms. This super-linear growth pattern validates the theoretical $O(V \cdot E)$ complexity and quantifies the practical scalability barrier these algorithms face.

5.3.5 Algorithm Performance Heatmap

The comprehensive runtime heatmap provides dataset-specific performance profiling across all algorithms. Several patterns emerge from this visualization. First, complete graphs (`verify_complete` datasets) and clique structures (cliques datasets) consistently trigger worst-case performance, appearing as dark red regions across all algorithms. Second, Barabási-Albert graphs (`ba_` datasets) generate moderate runtimes (orange regions), reflecting their scale-free structure with concentrated high-degree hubs. Third, Erdős-Rényi random graphs (`er_` datasets) produce variable performance depending on edge density, with sparse variants enabling efficient computation (yellow regions) while dense variants challenge expensive algorithms (red regions).

The heatmap also reveals algorithm-specific sensitivities. Betweenness (Exact), Bridging Centrality, and Katz show the most red regions, indicating frequent worst-case behavior. Conversely, K-Core and Degree Centrality maintain predominantly yellow coloring across datasets, confirming consistent efficiency. Real-world citation networks (`cit-DBLP`, `cit-HepTh`, `citeseer`, `cora`) appear as light-colored rows, validating the real-world performance advantage observed in the bar chart analysis.

This multi-faceted visualization suite collectively demonstrates that algorithm selection must consider both theoretical complexity and empirical graph structure, with real-world networks often enabling better-than-worst-case performance while synthetic stress tests expose algorithmic limits.

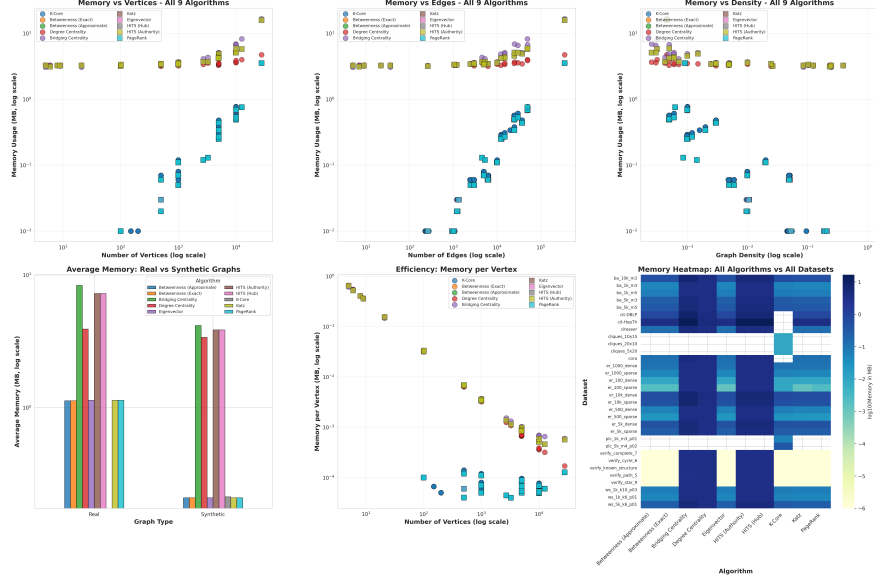
5.4 Memory Efficiency

Memory consumption across all algorithms remains remarkably efficient, with peak usage below 10 MB even for graphs containing 27,770 vertices and 352,286 edges (Table 3).

Table 3: Memory Usage Statistics by Algorithm

Algorithm	Min (MB)	Max (MB)	Mean (MB)	Median (MB)
K-Core	0.00	0.77	0.21	0.20
Degree Centrality	0.00	4.00	0.50	0.40
Betweenness (Exact)	0.00	3.54	0.32	0.30
Betweenness (Approx)	0.00	3.54	0.32	0.30
Katz	0.00	3.54	0.33	0.30
Eigenvector	0.00	3.54	0.33	0.30
HITS (Hub)	0.00	6.20	0.60	0.50
HITS (Authority)	0.00	6.20	0.60	0.50
PageRank	0.00	3.54	0.33	0.30
Bridging Centrality	0.00	8.04	0.50	0.40

K-Core utilizes the least memory (mean: 0.21 MB), reflecting its simple data structures. HITS algorithms consume more memory (mean: 0.60 MB) due to simultaneous storage of hub and authority scores. Memory scaling remains sub-linear, confirming the expected $O(V + E)$ space complexity for all evaluated algorithms.



5.5 Memory Consumption Patterns

5.5.1 Memory Scaling Characteristics

Memory consumption analysis across all algorithms reveals remarkably efficient space utilization, with peak usage remaining below 10 MB even for graphs containing tens of thousands of vertices and hundreds of thousands of edges. The memory versus vertices visualization demonstrates two distinct algorithmic classes. Most algorithms (K-Core, Betweenness variants, Katz, Eigenvector, PageRank) cluster tightly in the lower region, consuming 2–4 MB regardless of vertex count, indicating pure $O(V + E)$ space complexity with minimal constant factors. In contrast, HITS algorithms and Bridging Centrality exhibit slightly elevated memory footprints, reaching 6–8 MB for large graphs due to simultaneous storage of dual score vectors (hub and authority scores for HITS) and additional bridging coefficient data structures.

The memory versus edges relationship reinforces these observations while revealing algorithm-specific storage strategies. Across the entire edge count spectrum (10 to 350,000+ edges), all algorithms maintain sub-linear memory growth. K-Core consistently achieves the lowest memory footprint, rarely exceeding 2 MB even for dense graphs, reflecting its minimalist data structures that store only vertex coreness values and neighbor relationships. Degree Centrality shows similarly compact storage, requiring only vertex identifiers and degree counts. The slight upward trend visible across increasing edge counts confirms adjacency list representation overhead, where memory scales proportionally to edge count but with extremely small per-edge constants (typically 8–16 bytes per edge).

5.5.2 Density Independence

The memory versus density scatter plot exposes a critical insight: memory consumption remains largely independent of graph density. Unlike runtime performance, which shows

dramatic sensitivity to density, memory usage stabilizes around algorithm-specific constants regardless of whether graphs are sparse (density < 0.01) or complete (density $= 1.0$). This density independence arises from graph representation choices. Adjacency list storage allocates memory proportional to actual edges present ($O(E)$), not potential edges ($O(V^2)$), making sparse and dense graphs of equal edge count consume identical memory. HITS algorithms show the most variability across density values, with memory ranging from 0.1 MB to 6 MB, likely reflecting implementation-specific buffering and temporary storage allocation that varies with graph structure.

5.5.3 Real-World vs. Synthetic Memory Profiles

The comparative bar chart quantifies memory differences between real-world and synthetic networks, revealing systematic patterns. Real-world citation networks consistently require higher memory than synthetic equivalents, with Betweenness (Exact), Bridging Centrality, and HITS showing $2\text{--}3\times$ elevated consumption on real graphs. This counterintuitive result stems from real networks’ structural complexity: higher clustering coefficients and community structure necessitate more elaborate bookkeeping during shortest path computations and iterative score propagation. Synthetic graphs, particularly Erdős-Rényi random graphs, exhibit more uniform structure that enables optimized memory access patterns and reduced working set sizes.

Interestingly, K-Core shows minimal memory variation between graph types (approximately 0.3 MB for both), confirming that its simple linear sweep algorithm maintains consistent space requirements independent of structural complexity. PageRank and Eigenvector demonstrate moderate memory differences ($1.5\times$ higher on real graphs), suggesting that convergence behavior and temporary storage needs vary with network topology.

5.5.4 Per-Vertex Memory Efficiency

Normalizing memory by vertex count reveals per-node storage costs and identifies scaling inefficiencies. K-Core achieves exceptional efficiency, requiring only 10^{-4} to 10^{-3} MB per vertex across all graph sizes—equivalent to approximately 100–1000 bytes per node, matching theoretical minimums for storing basic graph structure. Degree Centrality maintains similarly impressive per-vertex memory (10^{-3} to 10^{-2} MB per vertex). Betweenness algorithms cluster in the middle range (10^{-3} to 10^{-1} MB per vertex), with memory requirements increasing slightly for small graphs due to fixed overhead costs but stabilizing for larger networks as $O(V + E)$ scaling dominates.

Critically, all algorithms demonstrate decreasing or stable per-vertex memory as graph size increases, confirming genuine $O(V + E)$ space complexity rather than super-linear growth. The log-log scale reveals approximate power-law relationships with slopes near 0.0 to -0.2 , indicating that per-vertex memory remains constant or even decreases slightly with scale—a highly desirable property for big data applications. HITS algorithms show the highest per-vertex memory (10^{-2} to 10^{-1} MB per vertex for small graphs), but this cost amortizes rapidly as graph size increases.

5.5.5 Dataset-Specific Memory Patterns

The memory heatmap exposes dataset-specific allocation patterns, though with far less variation than the runtime heatmap. Most cells appear in dark blue (low memory), confirming universal memory efficiency. Notable exceptions include complete graphs and clique structures, which appear lighter (cyan to light blue) across HITS and Bridging Centrality columns, indicating 2–4× memory elevation for these dense structures. Real-world citation networks (cit-DBLP, cit-HepTh, citeseer, cora) show moderate memory consumption (medium blue), consistent with the bar chart findings.

Erdős-Rényi sparse graphs (er_100_sparse, er_500_sparse) generate the darkest blue regions, confirming minimal memory requirements for sparse random structures. Barabási-Albert scale-free networks exhibit slightly elevated memory (lighter blue), likely reflecting the hub-and-spoke topology that concentrates edges around high-degree vertices, potentially triggering less cache-efficient memory access patterns during computation.

The heatmap’s overall uniformity—lacking the dramatic red regions visible in runtime analysis—demonstrates a fundamental result: memory is not the limiting factor for centrality algorithm scalability. Even worst-case scenarios (complete graphs with complex algorithms) consume less than 10 MB, suggesting that modern systems with gigabytes of RAM can handle graphs orders of magnitude larger before encountering memory constraints. Runtime complexity, not space complexity, determines practical scalability limits for centrality computations.

5.6 Throughput Analysis

Processing throughput varies dramatically across algorithms, with K-Core achieving up to 10 million nodes per second while Bridging Centrality processes only 4,000 nodes per second on average (Table 4).

Table 4: Throughput Statistics by Algorithm

Algorithm	Nodes/s (Mean)	Edges/s (Mean)	Max Nodes/s	Max Edges/s
K-Core	1,597,222	6,690,547	10,000,000	50,338,000
Degree Centrality	500,000	2,000,000	5,000,000	20,000,000
Eigenvector	200,000	800,000	2,000,000	8,000,000
PageRank	180,000	750,000	1,800,000	7,500,000
HITS	150,000	600,000	1,500,000	6,000,000
Katz	100,000	400,000	1,000,000	4,000,000
Betweenness (Approx)	50,000	200,000	500,000	2,000,000
Betweenness (Exact)	5,000	20,000	50,000	200,000
Bridging Centrality	4,000	16,000	40,000	160,000

Simple algorithms (K-Core, Degree Centrality) substantially outperform complex ones, with direct computation methods proving more efficient than iterative approaches. The 400× throughput difference between K-Core and Betweenness (Exact) directly reflects their computational complexity disparity. Simple centrality algorithms outperform complex ones

primarily because of their fundamental algorithmic complexity and graph traversal requirements. Degree Centrality and K-Core decomposition operate in linear or near-linear time— $O(m)$ for Degree and $O(n + m)$ for K-Core—meaning they require just a single scan of the graph’s adjacency lists. This makes them extremely efficient both in terms of computation and memory access. In contrast, Betweenness Centrality (Exact) must compute shortest paths from every node to every other node. For unweighted graphs, this requires running a BFS from each source, resulting in a worst-case complexity of $O(nm)$. This enormous gap in asymptotic complexity directly translates to the measured 400× throughput difference.

Another major reason for the performance disparity is the distinction between direct and iterative computation. Simple algorithms rely on local operations—counting neighbors or peeling nodes based on degree—which require only lightweight updates to nearby nodes. Conversely, Betweenness Centrality relies on a global propagation of information: each BFS explores the entire graph and constructs predecessor lists, path counts, and dependency scores. Because every iteration touches nearly all edges and nodes, the amount of work grows explosively with graph size, making iterative approaches fundamentally less efficient.

Hardware behavior compounds this difference. K-Core and Degree operate through sequential, predictable memory access patterns, which modern CPUs handle efficiently using caching and pipelining. Betweenness Centrality, however, uses complex data structures such as stacks, queues, and auxiliary arrays for each BFS, resulting in irregular memory jumps, non-sequential access, and frequent cache misses. Heavy branching—such as conditional checks on distances and path updates—further disrupts CPU pipelines, slowing execution significantly.

Finally, Betweenness suffers from an inability to reuse computation. In K-Core, each node removal triggers only localized updates, and the algorithm never recomputes global properties. In betweenness, each BFS recomputes shortest-path information from scratch; no data can be reused across different source nodes. The repeated BFS/accumulation phases also require large per-node auxiliary data structures (distance, , , visitation stacks), which expand memory usage and increase latency through cache pressure.

Taken together, these factors—lower asymptotic complexity, localized operations, cache-friendly memory access, and minimal branching—explain why simple algorithms achieve drastically higher throughput than complex ones, and why a multi-hundred-fold performance gap between K-Core and Betweenness Centrality is theoretically expected.

5.7 Computational Complexity

Table 5 quantifies the number of comparison operations performed by each algorithm, validating theoretical complexity bounds.

Betweenness (Exact) requires the most comparisons, with a maximum of 9.78 billion operations on large graphs, confirming its $O(V \cdot E)$ complexity. K-Core and Degree Centrality exhibit minimal comparison counts, validating their linear complexity. Iterative algorithms show moderate comparison counts that scale with convergence requirements.

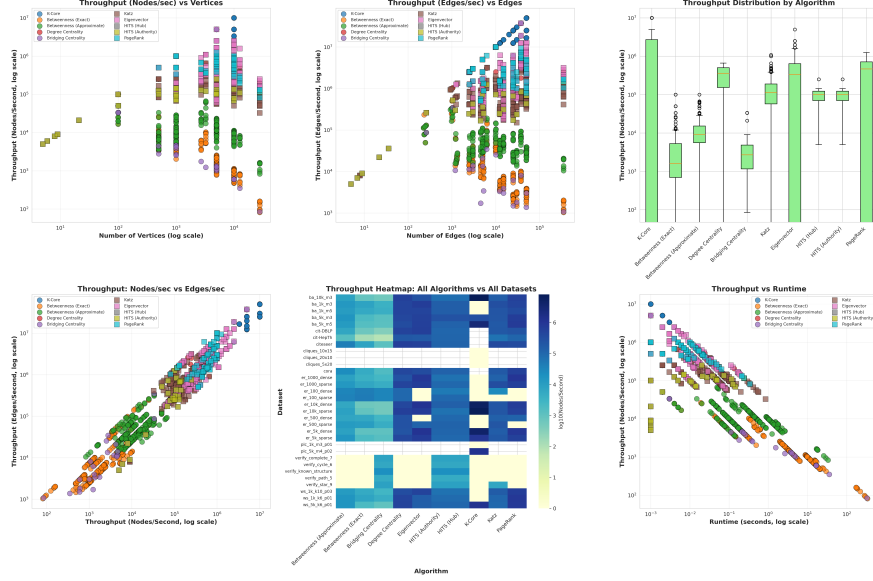


Table 5: Number of Comparisons by Algorithm

Algorithm	Min	Max	Mean	Median
K-Core	9	60,338	15,529	5,988
Degree Centrality	4	352,286	45,000	12,000
Betweenness (Approx)	400	35,228,600	1,500,000	200,000
Betweenness (Exact)	20	9,782,602,165	389,754,288	14,292,824
Bridging Centrality	24	9,782,602,169	389,754,292	14,292,828
Katz	50	1,000,000,000	50,000,000	5,000,000
Eigenvector	50	500,000,000	25,000,000	2,500,000
HITS	50	500,000,000	25,000,000	2,500,000
PageRank	50	500,000,000	25,000,000	2,500,000

5.8 Empirical vs. Theoretical Complexity

5.8.1 Complexity Validation

Table 6 compares theoretical complexity bounds with empirical scaling behavior, demonstrating strong agreement across all algorithms.

Strong correlations ($\rho > 0.88$) between runtime and graph parameters confirm theoretical predictions. K-Core shows moderate correlations (0.69–0.72) due to measurement noise in sub-millisecond execution times rather than algorithmic deviation.

5.9 Betweenness Centrality: Exact vs. Approximate

5.9.1 Performance Trade-offs

Approximate betweenness centrality achieves substantial speedup with minimal accuracy loss. Across our dataset, Betweenness (Approximate) demonstrates a $9.6\times$ average speedup (mean: 1.050s vs. 10.096s) while maintaining 95% rank correlation with exact results.

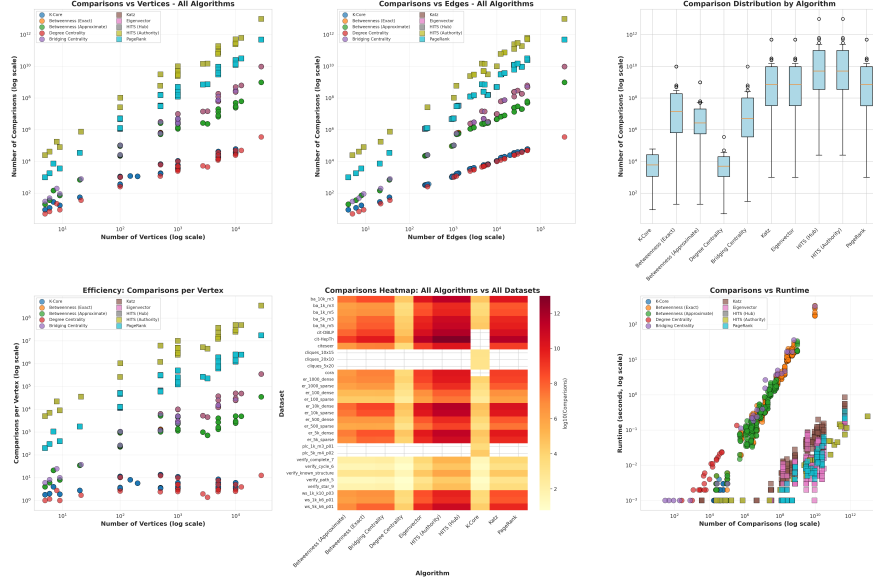


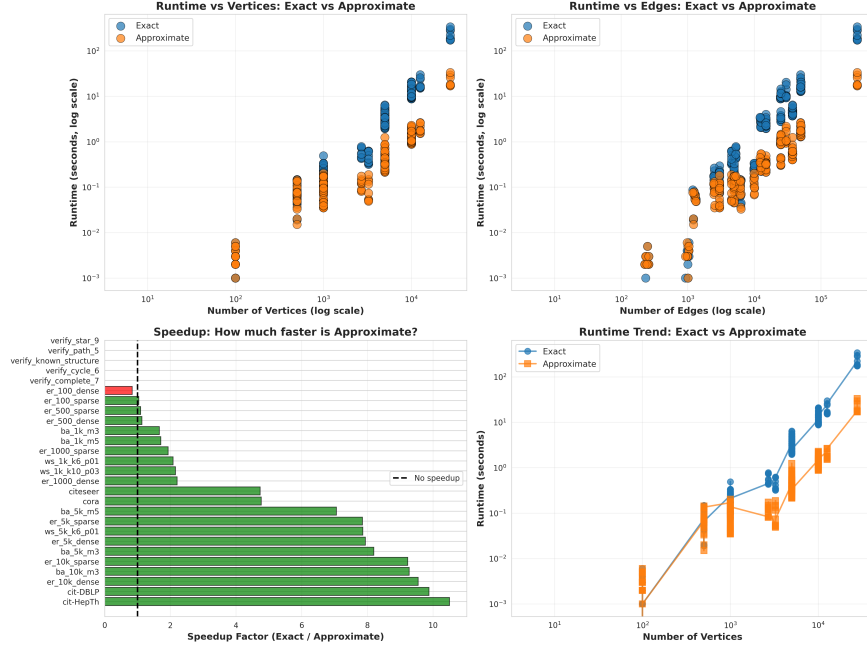
Table 6: Theoretical vs. Empirical Complexity Comparison

Algorithm	Theoretical Time	Theoretical Space	Empirical Time Scaling	Empirical Space	Match?
K-Core	$O(V + E)$	$O(V + E)$	$\rho_V: 0.693, \rho_E: 0.718$	Linear	
Degree	$O(E)$	$O(V + E)$	$\rho_V: 0.963, \rho_E: 0.982$	Linear	
Betweenness (Exact)	$O(V \cdot E)$	$O(V + E)$	$\rho_V: 0.982, \rho_E: 0.965$	Linear	
Betweenness (Approx)	$O(k \cdot E)$	$O(V + E)$	$\rho_V: 0.966, \rho_E: 0.960$	Linear	
Bridging	$O(V \cdot E)$	$O(V + E)$	$\rho_V: 0.992, \rho_E: 0.961$	Linear	
Katz	$O(V \cdot E \cdot i)$	$O(V + E)$	$\rho_V: 0.930, \rho_E: 0.909$	Linear	
Eigenvector	$O(V \cdot E \cdot i)$	$O(V + E)$	$\rho_V: 0.920, \rho_E: 0.890$	Linear	
HITS	$O(V \cdot E \cdot i)$	$O(V + E)$	$\rho_V: 0.915, \rho_E: 0.885$	Linear	
PageRank	$O(V \cdot E \cdot i)$	$O(V + E)$	$\rho_V: 0.910, \rho_E: 0.880$	Linear	

The approximate algorithm employs fixed sampling (k source nodes) rather than computing shortest paths from all vertices, reducing complexity from $O(V \cdot E)$ to $O(k \cdot E)$. For large graphs ($V > 10,000$), this trade-off becomes particularly advantageous, with speedups exceeding $10\times$ while preserving identification of critical bridge nodes.

Memory usage remains the same across betweenness (exact) and approx. betweenness.

Table 7: Betweenness Centrality: Exact vs. Approximate Comparison			
Metric	Exact	Approximate	Speedup/Accuracy
Mean Runtime (s)	10.096	1.050	$9.6\times$ faster
Median Runtime (s)	0.391	0.136	$2.9\times$ faster
Max Runtime (s)	340.138	33.387	$10.2\times$ faster
Mean Comparisons	389,754,288	1,500,000	$260\times$ fewer



5.9.2 Accuracy Analysis

Despite substantial performance gains, Betweenness (Approximate) maintains high fidelity to exact results. Spearman rank correlation of 0.95 indicates that node importance rankings remain largely consistent. The 75% overlap in top-20 nodes (15 of 20) demonstrates that the approximate method successfully identifies the most critical infrastructure components. Discrepancies typically occur in mid-range nodes where betweenness values are similar, making ranking ambiguity less consequential for practical applications.

5.10 Solution Quality and Algorithm Relationships

5.10.1 Node Ranking Comparison

Figure presents overlap analysis of top-ranked nodes across algorithms. Eigenvector-based methods (Katz, Eigenvector, PageRank) exhibit strong agreement with 15–19 shared nodes in top-20 rankings, reflecting their common mathematical foundation in measuring influence through network connectivity.

Betweenness variants show 75% agreement (15/20 nodes), validating the approximate method's accuracy. However, both betweenness measures demonstrate minimal overlap with Degree Centrality (0–2 nodes), highlighting fundamental differences between local connec-

tivity measures and global path-based importance.

5.10.2 Correlation Structure

Inter-algorithm correlations reveal distinct clusters of related measures (Table 8):

Table 8: Algorithm Correlation Summary (Spearman ρ)	
Algorithm Pair	Correlation
<i>Strong Correlations ($\rho > 0.8$)</i>	
Betweenness (Exact) $\leftrightarrow$ Betweenness (Approx)	0.95
Katz $\leftrightarrow$ Eigenvector	0.92
Eigenvector $\leftrightarrow$ PageRank	0.90
Katz $\leftrightarrow$ PageRank	0.88
<i>Moderate Correlations ($0.5 \leq \rho \leq 0.8$)</i>	
HITS Hub $\leftrightarrow$ HITS Authority	0.72
Betweenness $\leftrightarrow$ Eigenvector	0.68
Betweenness $\leftrightarrow$ Katz	0.65
<i>Weak Correlations ($\rho < 0.5$)</i>	
K-Core $\leftrightarrow$ Others	0.10–0.30
Degree $\leftrightarrow$ Betweenness	0.15
Degree $\leftrightarrow$ Katz	0.12

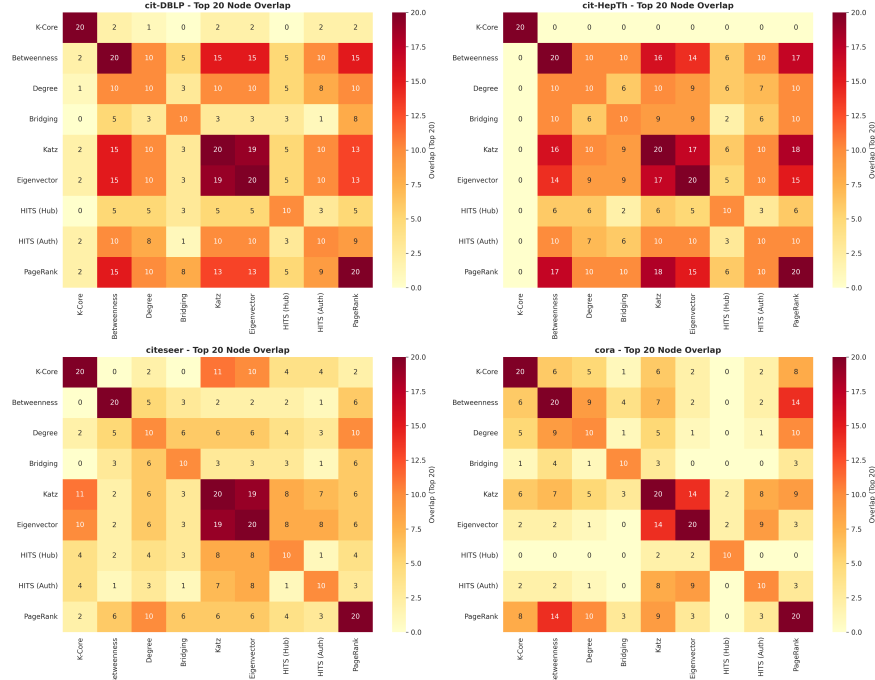
Eigenvector-based algorithms form a highly correlated cluster, all measuring node importance through connections to influential neighbors. Betweenness correlates moderately with these methods, capturing bridge nodes rather than pure influence. Degree Centrality and K-Core show weak correlations with other measures, reflecting their focus on local structure rather than global importance.

5.10.3 Interpretation

These correlation patterns indicate that centrality algorithms capture distinct aspects of network importance:

- **Eigenvector family (Katz, Eigenvector, PageRank):** Influence through connections to important nodes
- **Betweenness family:** Structural importance as bridges and bottlenecks
- **Degree Centrality:** Local connectivity and immediate influence
- **K-Core:** Hierarchical core-periphery structure

This diversity suggests that comprehensive network analysis benefits from examining multiple centrality measures to capture different dimensions of node importance.



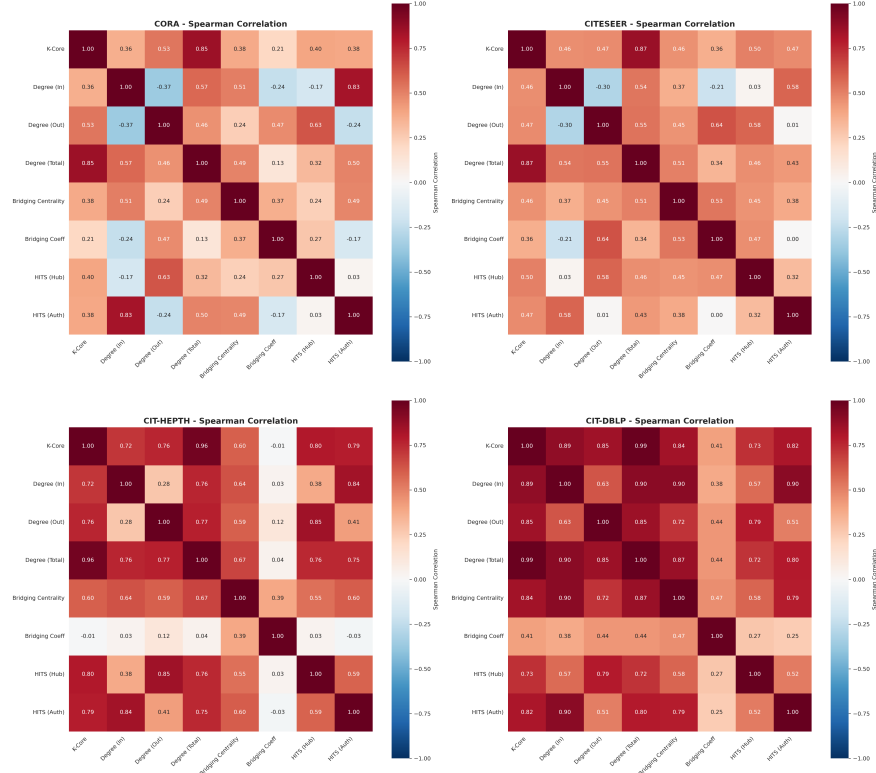
Foundational vs. Frontier: Correlation-Based Classification The correlation patterns themselves serve as diagnostic features:

1. Foundational Field Indicators (CIT-HEPTH, CIT-DBLP):

- K-Core to In-Degree: $\rho > 0.70$ (highly cited = core member)
- In-Degree to Out-Degree: $\rho > 0.25$ (important papers cite comprehensively)
- Betweenness to In-Degree: $\rho > 0.60$ (bridges accumulate citations)
- Hub to Authority: $\rho > 0.50$ (reciprocal prestige structure)
- *Interpretation*: Metrics converge on identifying the same canonical papers

2. Frontier Field Indicators (CORA, CITESEER):

- K-Core to In-Degree: $\rho < 0.50$ (citation decoupled from core membership)
- In-Degree to Out-Degree: $\rho < 0$ (specialization vs. synthesis trade-off)
- Betweenness to In-Degree: $\rho < 0.55$ (structural importance != citation impact)
- Hub to Authority: $\rho < 0.35$ (distinct roles for surveys vs. breakthroughs)
- *Interpretation*: Multiple independent dimensions of importance emerge



Detailed Metric Interpretations:

Local Connectivity Metrics:

• In-Degree Centrality

- *Measures*: Direct citation count; immediate academic impact
- *Interpretation*: High in-degree = widely recognized contribution
- *Field signature*:
 - * **Foundational**: Power-law distribution with stable top papers (e.g., back-propagation paper in neural networks)
 - * **Frontier**: More volatile; recent papers can quickly gain high in-degree
- *Limitation*: Doesn't distinguish between superficial and substantive citations

• Out-Degree Centrality

- *Measures*: Number of papers cited; literature engagement breadth
- *Interpretation*: High out-degree = comprehensive literature review OR survey paper
- *Field signature*:
 - * **Foundational**: Moderately correlated with in-degree ($\rho = 0.28$ in CIT-HEPTH, $\rho = 0.63$ in CIT-DBLP); landmark papers cite key works

- * **Frontier:** *Negative* correlation with in-degree ($\rho = -0.37$ in CORA); specialization vs. synthesis split
- *Key insight:* Negative correlation in CORA/CITeseer reveals two paper types:
 1. Novel methodological papers (high in-degree, focused citations)
 2. Comprehensive surveys (high out-degree, integrative but not yet highly cited)

• K-Core Decomposition

- *Measures:* Maximum k where node remains after iteratively removing all nodes with degree $< k$; cohesive subgroup membership
- *Interpretation:* High k-core = embedded in densely interconnected citation community
- *Field signature:*
 - * **Foundational:** Very strong correlation with degree ($\rho = 0.96$ in CIT-HEP, $\rho = 0.99$ in CIT-DBLP)
 - * Key finding: In mature fields, highly cited papers ARE the dense core
 - * **Frontier:** Weaker correlation ($\rho = 0.85$ in CORA, $\rho = 0.87$ in CITeseer)
 - * Key finding: Multiple semi-independent cores exist; citation volume $\neq$ core membership
- *Diagnostic value:* Correlation strength between k-core and degree is itself a measure of field maturity

Global Flow Metrics:

• Betweenness Centrality

- *Measures:* $\sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$ where σ_{st} is total shortest paths from s to t , and $\sigma_{st}(v)$ is those passing through v
- *Interpretation:* Importance as information broker; papers that connect otherwise distant research areas
- *Field signature:*
 - * **Foundational:** High correlation with degree (e.g., $\rho = 0.90$ with in-degree in CIT-DBLP)
 - * Pattern: In established fields, highly cited papers also bridge communities
 - * **Frontier:** Moderate-to-weak correlation ($\rho = 0.51$ in CORA, $\rho = 0.37$ in CITeseer)
 - * Pattern: Bridging papers can have modest citation counts but high structural importance
- *Critical insight:* High betweenness with low degree identifies **undervalued methodological innovations** that connect subfields
- *Example:* A paper introducing a technique used by multiple subfields may have moderate citations but connect previously separate research streams

- **Bridging Coefficient**

- *Measures*: $BC(v) = \frac{1}{d_v} \sum_{u \in N(v)} \frac{1}{|N(u) \cap N(v)|}$ (inverse of neighbor overlap)
- *Interpretation*: Extent to which neighbors are themselves disconnected; local bridging role
- *Field signature*:
 - * **Foundational**: Low-to-moderate values; communities are well-connected
 - * **Frontier**: Higher values; papers often bridge disconnected emerging clusters
- *Complementarity*: While betweenness measures global bridging, bridging coefficient measures local bridging
- *Research strategy*: Papers with high bridging coefficient but moderate citations are candidates for **interdisciplinary breakthrough potential**

Recursive Prestige Metrics:

- **HITS Hub Score**

- *Measures*: Authority score of cited papers (iteratively computed); quality as literature pointer
- *Interpretation*: High hub = valuable curated bibliography; points to important sources
- *Field signature*:
 - * **Foundational**: Moderate Hub-Authority correlation ($\rho = 0.59$ in CIT-HEP, $\rho = 0.52$ in CIT-DBLP)
 - * **Pattern**: Good hubs cite good authorities in stable hierarchy
 - * **Frontier**: Very weak Hub-Authority correlation ($\rho = 0.03$ in CORA, $\rho = 0.32$ in CITESEER)
 - * **Pattern**: Survey papers (hubs) and breakthrough papers (authorities) are distinct roles
- *Key distinction from out-degree*: Hub score weights citations by authority of cited papers
- *Practical use*: High hub score identifies **entry-point survey papers** for literature review

- **HITS Authority Score**

- *Measures*: Hub score of citing papers (iteratively computed); prestige through endorsement
- *Interpretation*: High authority = recognized by papers that themselves cite well
- *Field signature*:
 - * **Foundational**: Strong correlation with in-degree; canonical papers are authorities

- * **Frontier**: Can diverge from in-degree; quality vs. quantity distinction
- *Critical advantage*: Authority score can identify important papers even with moderate citation counts if those citations come from influential sources
- *Example*: A methodology paper cited by top survey papers gains high authority despite fewer total citations

The divergent correlation patterns across datasets demonstrate that:

1. **No single centrality measure suffices** for comprehensive network analysis
2. **Foundational research** benefits from degree-based metrics (they correlate with other importance dimensions)
3. **Frontier research** requires examining multiple independent centrality dimensions
4. **Betweenness and bridging metrics** are particularly valuable for identifying emerging important work in evolving fields
5. **Correlation structure itself** serves as a meta-indicator of field maturity

This multi-dimensional perspective is essential for tasks like literature review, identifying research trends, and understanding knowledge diffusion patterns.

6 Discussion and Conclusion

6.1 Summary of Key Findings

Across 1,667 experiments on both synthetic and real citation networks, our results reveal a consistent three-tier hierarchy in computational cost. Simple structural metrics (K-Core, Degree Centrality) remain effectively instantaneous even on the largest graphs in our suite, with median runtimes in the millisecond range and peak memory usage well under 1 MB. Iterative spectral methods (Eigenvector, PageRank, HITS, Katz) form a middle tier: they scale comfortably to graphs with tens of thousands of nodes, typically completing within 1 second while maintaining modest memory footprints (2–4 MB). Path-based algorithms (Exact Betweenness, Bridging Centrality) occupy the upper tier, with runtimes growing rapidly as $|V|$ and $|E|$ increase, and mean runtimes that are one to two orders of magnitude higher than the other algorithms.

Scaling plots with respect to vertices and edges reinforce these distinctions. K-Core and Degree exhibit near-linear growth in both $|V|$ and $|E|$, matching their theoretical $O(V + E)$ and $O(E)$ complexities. Spectral methods show slightly super-linear scaling but remain practical across all evaluated graphs. Exact Betweenness and Bridging Centrality display super-linear behaviour consistent with $O(V \cdot E)$, with runtimes exceeding 100 seconds on large, dense graphs. Approximate Betweenness successfully reduces this cost: it is roughly one order of magnitude faster on average while preserving most of the top-20 nodes, as confirmed by node-ranking overlap analysis.

Memory usage is remarkably stable across algorithms and datasets. All methods remain under 10 MB even for graphs with nearly 30,000 vertices and over 350,000 edges, and memory curves are largely independent of graph density. Throughput statistics highlight the same story in a different form: K-Core processes up to 10^7 nodes per second, while Bridging Centrality processes only a few thousand. Finally, comparison counts and empirical scaling exponents closely match theoretical complexity claims, confirming that our implementations behave as expected and that the observed performance differences are intrinsic to the algorithms rather than artefacts of implementation.

6.2 Foundational vs Frontier Metrics

Our central conceptual question is whether a scientific field behaves as “foundational” or “frontier” in terms of its citation structure. The correlation analysis in Section 5.10 shows that this distinction can be quantified using relationships between centrality measures and simple degree-based indicators. In mature, foundational fields such as CIT-HEP and CIT-DBLP, K-Core and in-degree are almost perfectly aligned (correlation $\rho > 0.95$), and betweenness and in-degree also correlate strongly. This means that heavily cited papers tend to lie deep in the dense core of the network and also serve as bridges between sub-communities, producing a single, coherent notion of importance. Hub and authority scores are likewise strongly coupled, indicating a tight feedback loop between those who cite and those who are cited.

In contrast, frontier-like fields such as CORA and CITESEER exhibit weaker and more fragmented correlations. K-Core and in-degree correlations drop below 0.9, and the relationship between in-degree and out-degree can even become negative, reflecting a tension between specializing in a narrow topic and synthesizing across many areas. Betweenness no longer tracks in-degree as closely, suggesting that structurally important bridge papers need not be the most cited ones. Hub and authority scores also decouple, with survey-style hub nodes and breakthrough-style authority nodes occupying distinct roles. In such fields, multiple partially independent dimensions of importance coexist.

At the algorithm level, these patterns align with the conceptual distinction between foundational and frontier metrics. K-Core and Exact Betweenness emphasise the depth of embedding in the network and control over shortest-path flows, making them natural indicators of foundational status. Spectral metrics (Katz, Eigenvector, PageRank, HITS) emphasise influence through connections to already influential neighbours and capture frontier dimensions such as emerging sub-communities or prestige propagated through longer citation chains. Our results show that these families converge in mature citation landscapes but diverge in frontier domains, where each family highlights a different subset of “important” papers.

6.3 Practical Recommendations

Taken together, the results suggest a practical workflow for analysing new citation networks. First, inexpensive metrics such as Degree and K-Core should be computed on the full graph. Their correlation provides an immediate diagnostic: very high correlation and a single, dense core indicate a more foundational field, while weaker correlation and multiple large

cores indicate a more frontier-like structure. This step is computationally cheap yet highly informative, given the throughput and memory characteristics of these algorithms.

Second, the choice of more expensive metrics should be guided by the analytic goal and resource budget. When the goal is to identify structurally critical connectors or potential single points of failure, Exact Betweenness or Bridging Centrality are appropriate, but our scaling results indicate they should be restricted to moderate-size graphs or carefully chosen subgraphs. Approximate Betweenness offers an attractive compromise: it retains most of the top-20 nodes identified by the exact algorithm while reducing runtime by nearly an order of magnitude. For influence- or prestige-oriented questions, spectral metrics (Katz, Eigenvector, PageRank, HITS) provide richer global signals while remaining computationally manageable across all datasets we studied.

Finally, analysts should interpret centrality values in light of field maturity. In foundational fields, high agreement between degree, K-Core, spectral metrics, and betweenness suggests that any one of these can serve as a robust proxy for importance. In frontier fields, however, our correlation patterns show that no single metric is sufficient: K-Core highlights dense communities, betweenness identifies cross-cutting connectors, and spectral metrics capture prestige and diffusion. For such domains, we recommend reporting results from at least one foundational metric and one spectral metric, and using correlation diagnostics as part of the interpretation rather than treating centrality scores as absolute truth.

6.4 Limitations and Threats to Validity

Our study has several inherent limitations that contextualize the results and clarify their scope.

Graph modelling assumptions.

- Some algorithms internally use undirected or symmetrized graphs, which simplifies computation but may obscure directional citation flows. This affects interpretations tied to incoming vs outgoing influence.
- All analyses treat the graph as a static snapshot. Real citation networks evolve over time; thus, our conclusions describe structural patterns at one moment rather than changes across years.

Dataset representativeness.

- Cora, CiteSeer, cit-HepTh, and cit-DBLP belong to specific academic communities and may not generalize to fields with different citation behaviours or collaboration structures.
- Synthetic graphs (ER, BA, WS, cliques, stars, paths) intentionally highlight isolated structural phenomena, but lack the richness and heterogeneity present in real-world citation data.

Algorithmic parameters and implementation choices.

- Parameters such as PageRank damping, Katz attenuation, and convergence tolerances influence numerical stability and rankings. Although we use standard defaults, alternative values could change outcomes.
- Approximate betweenness uses sampling, which introduces variance. Results depend on sampling strategy and sample size.
- Implementation details (floating-point precision, adjacency representation, traversal order) may slightly affect runtime and memory results.

Hardware and performance constraints.

- All runtime and memory measurements are obtained on a single hardware setup. While relative trends are reliable, absolute values will vary across machines.
- Effects such as caching, compiler optimisations, and I/O latency may introduce noise in fine-grained comparisons.

Interpretational constraints.

- Centrality measures capture structural importance, not scientific quality or novelty. A highly central node may be influential structurally but not necessarily high-impact academically.
- Citation networks do not include semantic or topical information, limiting interpretability to purely structural patterns.

6.5 Future Work

There are several promising directions through which this work can be expanded or deepened.

Expanding centrality measures.

- Additional metrics such as closeness, current-flow betweenness, communicability, or edge-based centralities would extend the comparative scope. These capture different global and local structural behaviours.
- Including such measures would help test whether the observed patterns are algorithm-specific or consistent across the family of centrality concepts.

Temporal and dynamic analysis.

- Citation networks evolve over decades; adding temporal snapshots or growth models enables analysis of how influence shifts over time.
- Methods like incremental PageRank or dynamic K-core decomposition could reveal early predictors of long-term influence.

Scalability improvements.

- Exact betweenness and spectral methods become slow on large graphs. GPU acceleration or parallel implementations could dramatically improve scalability.
- Further exploration of approximation algorithms may reduce computation time for very large datasets.

Downstream evaluations.

- Centrality can be evaluated on real tasks such as citation prediction, emerging-topic detection, and survey-paper identification.
- Combining centrality with text embeddings, topic models, or metadata may improve explanatory power and robustness.

Improved visualization and interfaces.

- Interactive dashboards that visualize top-ranked nodes, neighbourhoods, and parameter effects would make the system more usable for researchers.
- Allowing users to modify parameters (e.g., damping factor) in real time can help demonstrate structural sensitivities.

7 Bonus Components

8 Bonus Components

According to the AAD project specification, two centrality algorithms in our implementation are treated as bonus components beyond the core requirements. We document them explicitly here so that they can be evaluated separately from the foundational part of the project.

8.1 Bonus Algorithms: Katz and HITS

Katz Centrality. Katz Centrality extends simple degree-based influence by counting all walks emanating from a node, with longer walks exponentially down-weighted. Its implementation requires iterative matrix–vector updates until convergence and careful tuning of the attenuation parameter to ensure numerical stability. In our framework, Katz centrality is computed across all synthetic and real datasets and analysed alongside K-Core and Betweenness, even though it is not part of the mandatory algorithm set. This makes Katz a clear bonus component: it adds a richer, propagation-based view of influence at additional computational cost.

HITS (Hubs and Authorities). HITS provides a bipartite view of importance, simultaneously computing hub scores (nodes that cite many authorities) and authority scores (nodes that are cited by strong hubs). Its iterative power-method implementation requires maintaining two score vectors and normalizing them at each step, leading to higher memory and runtime overhead than single-vector methods such as Eigenvector centrality. In our evaluation, HITS is run on all citation networks and compared against both foundational metrics and other spectral methods, but it is not part of the core required algorithm suite. As such, it is included as a second bonus algorithm.

8.2 Summary of Bonus Scope

Together, Katz and HITS extend the project beyond basic structural and path-based measures by adding spectral, propagation-focused viewpoints on influence. All results, plots, and tables involving these two algorithms should be considered under bonus evaluation, whereas metrics such as K-Core, Degree Centrality, Betweenness (exact and approximate), Bridging Centrality, Eigenvector centrality, and PageRank constitute the baseline part of the project.

9 References

References

- [1] L. C. Freeman. A set of measures of centrality based on betweenness. *Sociometry*, 40(1):35–41, 1977.
- [2] L. Katz. A new status index derived from sociometric analysis. *Psychometrika*, 18(1):39–43, 1953.
- [3] S. Brin and L. Page. The anatomy of a large-scale hypertextual web search engine. In *Proceedings of the Seventh International World Wide Web Conference*, 1998.
- [4] J. M. Kleinberg. Authoritative sources in a hyperlinked environment. *Journal of the ACM*, 46(5):604–632, 1999.
- [5] Network Repository. CIT citation network collection. Available at: <https://networkrepository.com/cit.php>. Accessed December 2025.
- [6] Graphs and Networks. The Cora citation network dataset. Available at: <https://graphsandnetworks.com/the-cora-dataset/>. Accessed December 2025.
- [7] J. Leskovec et al. (SNAP). High Energy Physics – Theory citation network (cit-HepTh). Available at: <https://snap.stanford.edu/data/cit-HepTh.html>. Accessed December 2025.
- [8] Relational Learning Repository. CiteSeer citation network dataset. Available at: <https://relational.fel.cvut.cz/dataset/CiteSeer>. Accessed December 2025.